

Local ML Inference Deployment with Docker and Kubernetes

AI 545 / MLOps

1 Lecture Goal

In this demo lecture, we will deploy a simple machine learning inference service locally using Docker and Kubernetes. The goal is to go beyond theory and actually build a minimal but realistic MLOps deployment.

2 Project Structure

Create a new project directory and place the following files inside it.

```
ml-k8s-demo/  
train.py  
app.py  
requirements.txt  
Dockerfile  
deployment.yaml  
service.yaml
```

3 Step 1: Train and Save the Model

Create a file named `train.py`.

```
from sklearn.datasets import load_iris  
from sklearn.ensemble import RandomForestClassifier  
import joblib  
  
# Load data  
X, y = load_iris(return_X_y=True)  
  
# Train model  
model = RandomForestClassifier(n_estimators=50, random_state=42)  
model.fit(X, y)  
  
# Save model  
joblib.dump(model, "model.pkl")  
  
print("Model trained and saved as model.pkl")
```

Now create `requirements.txt`.

```
fastapi
uvicorn
scikit-learn
joblib
pydantic
```

Run the following commands locally.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python train.py
```

At this point, the file `model.pkl` should exist in the project directory. This file is the trained artifact that our inference API will load.

4 Step 2: Create the Inference API

Create a file named `app.py`.

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI(title="Iris Inference API")

model = joblib.load("model.pkl")

class IrisRequest(BaseModel):
    features: list[float]

@app.get("/")
def root():
    return {"message": "Iris inference service is running"}

@app.post("/predict")
def predict(data: IrisRequest):
    prediction = model.predict([data.features])[0]
    return {"prediction": int(prediction)}
```

This service is intentionally minimal. It loads the saved model once at startup and provides a prediction endpoint that expects a JSON body containing four numeric features.

5 Step 3: Test the API Locally Without Docker

Before containerizing the service, test it directly. This is an important debugging step because once the application moves into containers and then into Kubernetes, troubleshooting becomes harder.

Run the server locally.

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Open another terminal and test the endpoint.

```
curl http://127.0.0.1:8000/
```

Now test prediction.

```
curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{"features": [5.1, 3.5, 1.4, 0.2]}'
```

If the API works locally, stop the server and proceed.

6 Step 4: Create the Docker Image

Create a file named Dockerfile.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY model.pkl .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build the Docker image.

```
docker build -t iris-api:v1 .
```

Test the image locally before using Kubernetes.

```
docker run -p 8000:8000 iris-api:v1
```

In a second terminal, test it again.

```
curl http://127.0.0.1:8000/
```

```
curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{"features": [6.7, 3.1, 4.7, 1.5]}'
```

If that works, stop the container.

7 Step 5: Start the Local Kubernetes Cluster

Start Minikube.

```
minikube start
```

Verify the cluster is running.

```
kubectl get nodes
```

You should see one node in the Ready state. Even though it is one machine, Kubernetes still behaves like a real cluster. That is enough for this demo.

8 Step 6: Make the Docker Image Available to Minikube

Use the minikube build command to create the docker image.

Run the following command from the project directory.

```
minikube image build -t iris-api:v1 .
```

To verify the image exists inside Minikube, you may run:

```
minikube image ls
```

9 Step 7: Create the Kubernetes Deployment

Create a file named `deployment.yaml`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-api-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: iris-api
  template:
    metadata:
      labels:
        app: iris-api
    spec:
      containers:
        - name: iris-api
          image: iris-api:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
```

This Deployment tells Kubernetes to keep two replicas of our inference service running. A Deployment is the normal Kubernetes resource used to manage stateless application workloads. Kubernetes will create Pods from the template and keep the desired number running.

Apply the Deployment.

```
kubectl apply -f deployment.yaml
```

Inspect the Deployment.

```
kubectl get deployments
kubectl get pods
kubectl describe deployment iris-api-deployment
```

If something goes wrong, use logs.

```
kubectl logs <pod-name>
```

You can get the pod name first with:

```
kubectl get pods
```

10 Step 8: Expose the Deployment with a Service

Create a file named `service.yaml`.

```
apiVersion: v1
kind: Service
metadata:
  name: iris-api-service
spec:
  type: NodePort
  selector:
    app: iris-api
  ports:
    - port: 8000
      targetPort: 8000
      nodePort: 30080
```

Apply the Service.

```
kubectl apply -f service.yaml
```

Check the Service.

```
kubectl get services
kubectl describe service iris-api-service
```

A Service gives the application a stable endpoint and forwards traffic to matching Pods. In this demo, we use NodePort so that the app is reachable from the local machine through the Minikube node IP and a chosen port.

Access the service by:

```
minikube service iris-api-service --url
```

Then copy the returned URL and test the endpoint.

11 Step 9: Observe Kubernetes Behavior

Now that the application is running, use `kubectl` to observe the system.

```
kubectl get all
```

You can also watch pods live.

```
kubectl get pods -w
```

Describe one pod in detail.

```
kubectl describe pod <pod-name>
```

12 Step 10: Scale the Inference Service

Scale the Deployment from 2 replicas to 4 replicas.

```
kubectl scale deployment iris-api-deployment --replicas=4
```

Verify the new Pods.

```
kubectl get pods
kubectl get deployment iris-api-deployment
```

As request load grows, the same inference service can be replicated without changing the application code.

13 Step 11: Perform a Simple Update

Now simulate a small application update. Change the root message in `app.py` from:

```
return {"message": "Iris inference service is running"}
```

to:

```
return {"message": "Iris inference service v2 is running"}
```

Rebuild the image inside Minikube with a new tag.

```
minikube image build -t iris-api:v2 .
```

Update the Deployment image.

```
kubectl set image deployment/iris-api-deployment iris-api=iris-api:v2
```

Check rollout status.

```
kubectl rollout status deployment/iris-api-deployment
```

Test the root endpoint again.

```
curl http://$(minikube ip):30080/
```

You should see the new message. This demonstrates a rolling update workflow, where Kubernetes replaces Pods gradually rather than stopping everything at once.

14 Step 12: Useful Debugging Commands

When demos fail, the problem is usually one of a few things: the image is missing, the container crashed, the Service selector does not match the Pod labels, or the application is listening on the wrong port. The following commands are the first ones to use.

```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
```

15 Step 13: Clean Up

When the demo is over, delete the resources.

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
```

To stop the local cluster:

```
minikube stop
```

If you want to remove the cluster entirely:

```
minikube delete
```

16 References

1. Kubernetes Documentation: Overview. <https://kubernetes.io/docs/concepts/>
2. Kubernetes Documentation: Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
3. Kubernetes Documentation: Pods. <https://kubernetes.io/docs/concepts/workloads/pods/>
4. Kubernetes Documentation: Services. <https://kubernetes.io/docs/concepts/services-network/service/>
5. Kubernetes Tutorial: Deploying your first app on Kubernetes. <https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-app/deploy-intro/>

6. Kubernetes Tutorial: Viewing Pods and Nodes. <https://kubernetes.io/docs/tutorials/kubernetes-basics/explore/explore-intro/>
7. Kubernetes Tutorial: Exposing your app. <https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/>
8. Kubernetes Tutorial: Rolling updates. <https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/>
9. kubectl Reference. <https://kubernetes.io/docs/reference/kubectl/>
10. kubectl Quick Reference. <https://kubernetes.io/docs/reference/kubectl/quick-reference>
11. kubectl set image. https://kubernetes.io/docs/reference/kubectl/generated/kubectl_set/kubectl_set_image/
12. Minikube Start. <https://minikube.sigs.k8s.io/docs/start/>
13. Minikube image build. <https://minikube.sigs.k8s.io/docs/commands/image/>
14. Minikube image pushing and caching workflows. <https://minikube.sigs.k8s.io/docs/handbook/pushing/>
15. Docker Documentation. <https://docs.docker.com/>